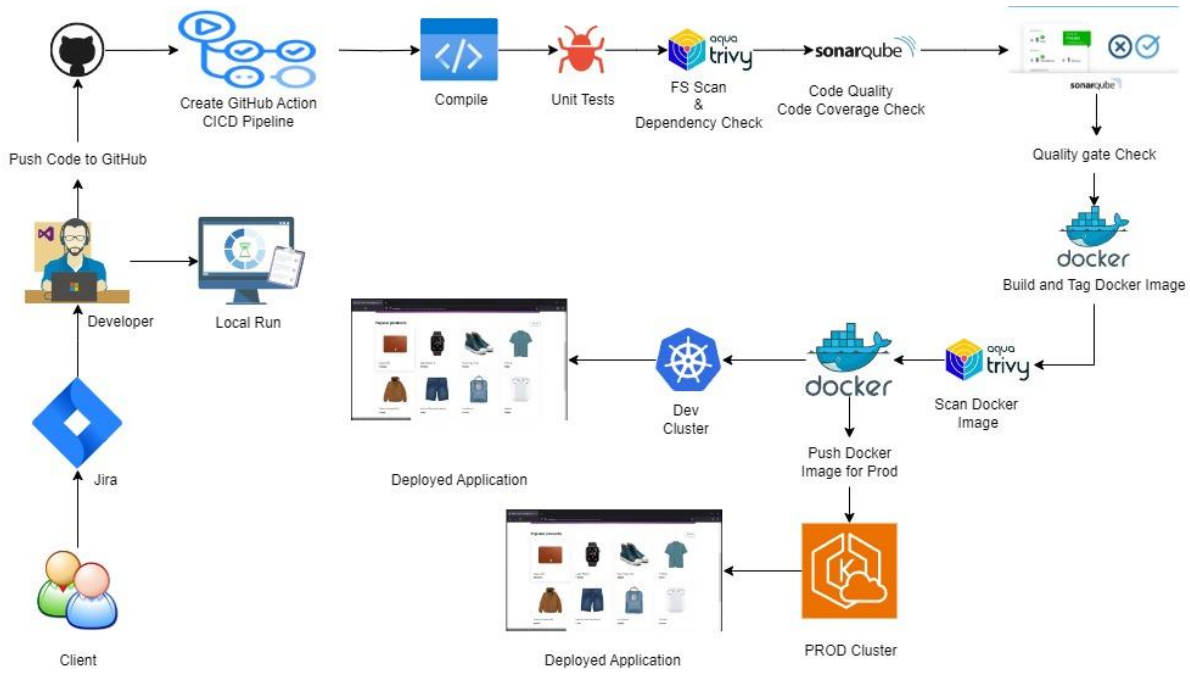
Building and Deploying an Application to Multiple Clusters



1. Introduction

This document outlines the process for building and deploying an application to multiple environments, including development and production clusters.

2. Workflow Overview

1. **Client Request:**
   - The client creates a Jira ticket for any updates or new features.
2. **Developer Process:**
   - **Local Development:**
     - The developer clones the repository to their local machine.
     - They implement the new feature or update.
     - The code is tested locally to ensure functionality.
   - **Code Push:**
     - The developer pushes the code to the `dev` branch on GitHub.
3. **Continuous Integration and Deployment (CI/CD):**
   - **GitHub Actions Pipeline:**
     - **Compilation:**
       - The pipeline triggers automatically on code push.
       - The code is compiled to check for syntax errors.
     - **Unit Testing:**
       - Unit tests are executed to verify the functionality of the application.

        - **Dependency Check:**
            - Aqua Trivy scans the application dependencies for vulnerabilities and outdated packages.
        - **Code Quality Check:**
            - SonarQube analyzes the code for bugs, vulnerabilities, and code smells.
            - Code coverage is assessed to ensure a minimum quality threshold.
        - **Build Artifacts:**
            - The application is built into an executable file (e.g., JAR for Java applications).
            - A Docker image is created and scanned with Aqua Trivy for vulnerabilities.
        - **Deployment to Development Cluster:**
            - The Docker image is pushed to Docker Hub.
            - YAML manifest files are used to deploy the application to the development cluster.
4. **Production Deployment:**
    - **Pre-Deployment:**
        - After successful deployment and verification in the development cluster, the Docker image is tagged (e.g., `prod-latest`).
    - **Production Deployment:**
        - The tagged Docker image is used to create new YAML manifest files for the production cluster.
        - The application is deployed to the production cluster.

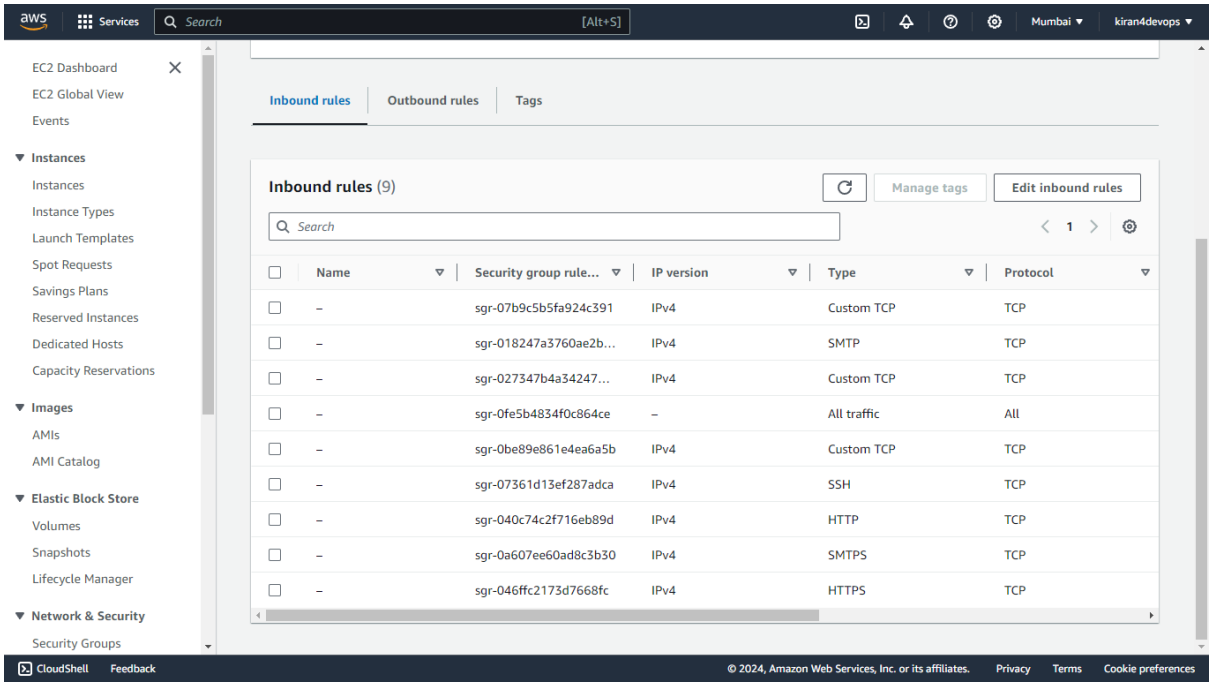
We need to set up two clusters: one for Production and one for Development.

- **Production Cluster:** We will set up an Amazon EKS cluster managed by AWS.
- **Development Cluster:** We will set up a self-hosted Kubernetes cluster.

Once both clusters are set up, we can start writing the CI/CD pipelines for each of them.

- **Open Required Ports:**

    - Ensure that the below mentioned necessary ports are open in your security group's inbound rules.



Creating a self hosted kubernetes cluster(Development Cluster)

- Create **Virtual Machines:**
  - **Master Node:**
    - Create a virtual machine with the following specifications:
      - **OS:** Ubuntu 20.04 LTS
      - **Size:** T2 medium
      - **Key Pair:** Use an existing key pair or create a new one. If creating a new key, name it and ensure the format is PEM.
    - **Worker Node:**
      - Create another virtual machine with the same specifications as the Master node.
- **Network Settings:**
  - Select the security group configured with the necessary open ports.
- **Storage:**
  - Set the storage size to 25 GB.
- **Launch VMs:**
  - Launch the virtual machines and wait for them to be in the running state. Name one as the Master node and the other as the Worker node.
- **Connect via SSH:**

- Connect to each machine via SSH(mobaxterm) to proceed with the Kubernetes setup.

Setting Up Master and Worker Nodes

Run the following commands on both the master and worker machines. Each command is explained below:

1. Update the package list:

```
sudo apt update
```

This command updates the list of available packages and their versions, ensuring you have the latest information on the newest versions and their dependencies.

2. Install Docker:

```
sudo apt install docker.io -y
```

This command installs Docker, a containerization platform, which is essential for running Kubernetes containers. The `-y` flag automatically confirms the installation.

3. Set permissions for Docker:

```
sudo chmod 666 /var/run/docker.sock
```

This command changes the permissions of the Docker socket file, allowing non-root users to run Docker commands.

4. Install transport packages and certificates:

```
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg
```

This command installs packages necessary for allowing the `apt` package manager to use HTTPS for retrieving packages. It also installs `curl` for downloading files and `gnupg` for handling encryption and signing.

5. Create a directory for the Kubernetes keyring:

```
sudo mkdir -p -m 755 /etc/apt/keyrings
```

This command creates the `/etc/apt/keyrings` directory with permissions set to 755, which is required for storing the Kubernetes signing key.

6. Download and store the Kubernetes signing key:

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | sudo
gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

This command downloads the Kubernetes signing key and converts it to the GPG format, saving it in the `/etc/apt/keyrings/kubernetes-apt-keyring.gpg` file. This key is used to verify the authenticity of Kubernetes packages.

7. Add the Kubernetes repository to the package list:

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg]
https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee
/etc/apt/sources.list.d/kubernetes.list
```

This command adds the Kubernetes repository to the list of sources from which `apt` can install packages. It specifies that packages from this repository should be verified using the previously downloaded GPG key.

8. Update the package list again:

```
sudo apt update
```

This command refreshes the package list to include the newly added Kubernetes repository.

9. Install Kubernetes components:

```
sudo apt install -y kubeadm=1.30.0-1.1 kubelet=1.30.0-1.1 kubectl=1.30.0-
1.1
```

This command installs specific versions of `kubeadm`, `kubelet`, and `kubectl`, which are essential components for setting up and managing a Kubernetes cluster. The `-y` flag confirms the installation automatically.

Setting Up the Master Node

1. Initialize Pod Network and Generate Token to Add Worker Node:

Run the following command on **master node** to initialize the Kubernetes master node with a specific pod network CIDR:

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
```

This command initializes the Kubernetes control plane on the master node and sets the pod network CIDR to `192.168.0.0/16`. It also generates a token that will be used by the worker nodes to join the cluster.

2. Note the `kubeadm join` Command:

After running the above command, a `kubeadm join` command will be provided. This command must be run on the **worker node(s)** to join them to the master node.

Joining Worker Nodes to the Master Node

Run the `kubeadm join` Command on the Worker Node(s):

Use the `kubeadm join` command generated by the master node initialization process on worker node to join them to the cluster. The command will look similar to this:

```
sudo kubeadm join <master-node-ip>:<port> --token <token> --discovery-token-ca-cert-hash <hash>
```

3. Set Up kubeconfig for kubectl Access:

Run the following commands on the **master node** to set up the kubeconfig file, which is required for `kubectl` to interact with the Kubernetes cluster:

1. **Create the .kube directory:**

   ```
   mkdir -p $HOME/.kube
   ```

2. **Copy the admin.conf file to the kubeconfig directory:**

   ```
   sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
   ```

3. **Change the ownership of the kubeconfig file:**

   ```
   sudo chown $(id -u):$(id -g) $HOME/.kube/config
   ```

   This command changes the ownership of the `config` file to the current user, allowing `kubectl` to use it for cluster management.

Setting Up Calico for Networking

Run the following command to set up Calico as the network plugin for your Kubernetes cluster:

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.24.0/manifests/calico.yaml
```

This command applies the Calico manifest, which configures the Calico network plugin in your Kubernetes cluster, providing networking and network policy capabilities.

Setting Up Ingress-nginx Controller

Run the following command to set up the Ingress-nginx controller:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v0.49.0/deploy/static/provider/baremetal/deploy.yaml
```

This command applies the Ingress-nginx controller manifest, which sets up the Ingress controller to manage ingress resources and route external traffic to your Kubernetes services.

Testing the Kubernetes Cluster

To verify that your Kubernetes cluster is operational, you can check the status of the nodes in the cluster using the following command:

1. Check the Status of Nodes:

```
kubectl get nodes
```

This command lists all the nodes in your Kubernetes cluster along with their status. If everything is set up correctly, you should see the master and worker nodes listed with their respective statuses (e.g., Ready).

- **Master Node**: This node should show a Ready status, indicating it is properly initialized and operational.
- **Worker Nodes**: These nodes should also show a Ready status if they have successfully joined the cluster.

Setting Up the Production EKS Cluster

We need to set up the Production cluster, which will be an Amazon EKS cluster. For creating the EKS cluster, we will use Terraform.

1. Create a Virtual Machine for Terraform:

To run Terraform commands, we need a dedicated virtual machine. Although you can run Terraform from your local machine, using a separate virtual machine is often more convenient.

- **Virtual Machine Details:**
  - **Name:** terraform server
  - **OS Version:** Ubuntu 24.04
  - **Size:** T2 medium
  - **Storage:** 25 GB
  - **Security Group:** Use the existing security group.

Create and launch the virtual machine, and wait for it to come online.

Once the virtual machine is ready, you can proceed with running Terraform commands to set up the EKS cluster

Connecting to the Virtual Machine and Setting Up AWS CLI

1. Connect to the Virtual Machine:

- **Connect to the virtual machine using SSH.**
- **Rename the machine as `Terraform` after connecting.**

2. Update the System:

Run the following command to update the package list:

```
sudo apt update
```

3. Install AWS CLI:

To interact with AWS resources, we need to install the AWS CLI. Execute the following commands to install AWS CLI:

```
sudo apt install awscli -y
```

4. Configure AWS CLI:

Run the following command to configure the AWS CLI and provide your AWS credentials:

```
aws configure
```

You will be prompted to enter the following information:

1. **Access Key ID:**
   - To get the Access Key ID, you can create an IAM user in the AWS Management Console with appropriate permissions and generate access keys for that user.
   - For demo purposes, you can use the root user credentials, although this is not recommended for production environments.
   - Navigate to the AWS Management Console, go to your profile, select **Security Credentials**, and create a new access key. Copy the Access Key ID.
2. **Secret Access Key:**
   - This key is generated along with the Access Key ID. Copy the Secret Access Key from the same page where you obtained the Access Key ID.
3. **Default Region Name:**
   - Enter the region closest to you. For example, use `ap-south-1` for Mumbai.
4. **Default Output Format:**
   - You can press **Enter** to accept the default format or specify your preferred format.

After entering the required details, the AWS CLI will be configured, and you will be connected to your AWS account.

Setting Up Terraform for EKS Cluster Creation

1. Install Terraform:

Before creating an EKS cluster on AWS, ensure that Terraform is installed on your machine. Run the following command to check if Terraform is already installed:

```
terraform --version
```

If Terraform is not present, you will need to install it. Use the following command to install Terraform via Snap:

```
sudo snap install terraform --classic
```

Make sure to include the `--classic` flag; otherwise, you may encounter errors.

2. Verify Installation:

After installation, verify that Terraform is installed correctly by running:

```
terraform --version
```

3. Prepare Terraform Files:

To execute Terraform commands, you need the appropriate Terraform configuration files. Make sure you have these files prepared before running Terraform commands to create the EKS cluster.

Here is the generalized Terraform configuration files (please update according to your environment) for setting up an EKS cluster on AWS, including both the VPC and the necessary resources for the cluster and nodes.

Feel free to adjust the names, regions, and any other parameters according to your specific requirements.

- **Create the `main.tf` file:**
  - First, create a directory for your EKS configuration:

    ```
    mkdir -p EKS
    cd EKS
    ```

  - Create the `main.tf` file:

    ```
    vi main.tf
    ```

  - Paste the following contents into `main.tf` and save the file:

    ```
    provider "aws" {
      region = "us-east-1"  # Change this to your preferred region
    }
    ```

```
resource "aws_vpc" "my_vpc" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "my-vpc"
  }
}

resource "aws_subnet" "my_subnet" {
  count                   = 2
  vpc_id                  = aws_vpc.my_vpc.id
  cidr_block              = cidrsubnet(aws_vpc.my_vpc.cidr_block, 8,
count.index)
  map_public_ip_on_launch = true
  availability_zone       = element(["us-east-1a", "us-east-1b"],
count.index)  # Change these AZs as needed

  tags = {
    Name = "my-subnet-${count.index}"
  }
}

resource "aws_internet_gateway" "my_igw" {
  vpc_id = aws_vpc.my_vpc.id

  tags = {
    Name = "my-igw"
  }
}

resource "aws_route_table" "my_route_table" {
  vpc_id = aws_vpc.my_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.my_igw.id
  }

  tags = {
    Name = "my-route-table"
  }
}

resource "aws_route_table_association" "a" {
  count          = 2
  subnet_id      = aws_subnet.my_subnet[count.index].id
  route_table_id = aws_route_table.my_route_table.id
}

resource "aws_security_group" "my_cluster_sg" {
  vpc_id = aws_vpc.my_vpc.id

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
```

```
    Name = "my-cluster-sg"
  }
}

resource "aws_security_group" "my_node_sg" {
  vpc_id = aws_vpc.my_vpc.id

  ingress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "my-node-sg"
  }
}

resource "aws_eks_cluster" "my_cluster" {
  name     = "my-cluster"
  role_arn = aws_iam_role.my_cluster_role.arn

  vpc_config {
    subnet_ids         = aws_subnet.my_subnet[*].id
    security_group_ids = [aws_security_group.my_cluster_sg.id]
  }

  depends_on =
[aws_iam_role_policy_attachment.my_cluster_role_policy]
}

resource "aws_eks_node_group" "my_node_group" {
  cluster_name    = aws_eks_cluster.my_cluster.name
  node_group_name = "my-node-group"
  node_role_arn   = aws_iam_role.my_node_group_role.arn
  subnet_ids      = aws_subnet.my_subnet[*].id

  scaling_config {
    desired_size = 3
    max_size     = 3
    min_size     = 3
  }

  instance_types = ["t2.medium"]

  remote_access {
    ec2_ssh_key = var.ssh_key_name
    source_security_group_ids = [aws_security_group.my_node_sg.id]
  }

  depends_on = [

aws_iam_role_policy_attachment.my_node_group_role_policy_attachment,
```

```
    aws_iam_role_policy_attachment.my_node_group_cni_policy,
    aws_iam_role_policy_attachment.my_node_group_registry_policy
  ]
}

resource "aws_iam_role" "my_cluster_role" {
  name = "my_cluster_role"

  assume_role_policy = <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "eks.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
EOF
}

resource "aws_iam_role_policy_attachment" "my_cluster_role_policy" {
  role       = aws_iam_role.my_cluster_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
}

resource "aws_iam_role" "my_node_group_role" {
  name = "my_node_group_role"

  assume_role_policy = <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
EOF
}

resource "aws_iam_role_policy_attachment"
"my_node_group_role_policy_attachment" {
  role       = aws_iam_role.my_node_group_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
}

resource "aws_iam_role_policy_attachment" "my_node_group_cni_policy"
{
  role       = aws_iam_role.my_node_group_role.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
}
```

```
resource "aws_iam_role_policy_attachment"
"my_node_group_registry_policy" {
  role       = aws_iam_role.my_node_group_role.name
  policy_arn =
"arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
}
```

- Create **the `output.tf` file:**

  - Create the `output.tf` file:

    ```
    vi output.tf
    ```

  - Paste the following contents into `output.tf` and save the file:

    ```
    output "vpc_id" {
      description = "The ID of the VPC"
      value       = aws_vpc.my_vpc.id
    }

    output "subnet_ids" {
      description = "The IDs of the subnets"
      value       = aws_subnet.my_subnet[*].id
    }

    output "eks_cluster_id" {
      description = "The ID of the EKS cluster"
      value       = aws_eks_cluster.my_cluster.id
    }

    output "eks_node_group_id" {
      description = "The ID of the EKS node group"
      value       = aws_eks_node_group.my_node_group.id
    }
    ```

- **Create the `variables.tf` file:**

  - Create the `variables.tf` file:

    ```
    vi variables.tf
    ```

  - Paste the following contents into `variables.tf` and save the file:

    ```
    variable "ssh_key_name" {
      description = "The name of the SSH key pair to use for the
    instances"
      type        = string
      default     = "my_projects_key"
    }
    ```

First, ensure all three files (`main.tf`, `output.tf`, and `variables.tf`) are created.

1. **Initialize Terraform:** Run the command:

   ```
   terraform init
   ```

This command initializes the working directory, downloads necessary plugins, and prepares Terraform to execute commands.

2. **Apply Terraform Configuration:** Execute:

   ```
   terraform apply
   ```

   Terraform will prompt you to confirm the actions it plans to take. You will see a message asking if you want to perform these actions. Type `yes` to proceed.

   Alternatively, you can use:

   ```
   terraform apply -auto-approve
   ```

   This option automatically approves the execution without prompting for confirmation.

   Terraform will then start creating resources such as virtual machines, clusters, VPCs, subnets, etc. This process typically takes 5 to 10 minutes.

We can start writing the pipelines for our development cluster. First, we'll create a new branch in our repository:

1. **Create a New Branch:**
   - Name the new branch `dev` (or any name of your choice).
   - Create this branch from `main`.
2. **Set Up GitHub Actions Runner:**
   - Go to your repository's settings.
   - Navigate to **Actions** > **Runners**.
   - Remove any existing runner (you may need to provide MFA if enabled).

   To add a new runner:

   - Click **New Runner**.
   - Choose the appropriate runner type (e.g., Linux).
   - Follow the provided instructions to configure the runner on a Linux machine.
3. **Create a Virtual Machine for the Runner:**
   - Launch a new EC2 instance.
   - Name the instance `Runner`.
   - Use the latest Ubuntu 24.04 LTS version.
   - Select instance size (e.g., `t2.large` or `t2.medium`).
   - Assign the existing key pair and configure the security group.
   - Set the storage size (e.g., 30 GB).
   - Click **Launch Instance** and wait for it to start.
4. **Connect to the VM:**
   - Once the VM is running, connect to it via SSH.
   - Rename the instance to `Runner` and adjust any settings as needed.

This VM will act as a runner for GitHub Actions, allowing you to perform CI/CD tasks.

Connect to the VM via SSH. First, always remember to run:

```
sudo apt update
```

Meanwhile, let's review the steps to set up the runner on Linux.

Download and Configure the GitHub Actions Runner

1. **Create a Folder for the Runner:**

   ```
   mkdir actions-runner && cd actions-runner
   ```

2. **Download the Latest Runner Package:**

   ```
   curl -o actions-runner-linux-x64-2.317.0.tar.gz -L
   https://github.com/actions/runner/releases/download/v2.317.0/actions-
   runner-linux-x64-2.317.0.tar.gz
   ```

3. **Optional: Validate the Package Hash:**

   ```
   echo
   "9e883d210df8c6028aff475475a457d380353f9d01877d51cc01a17b2a91161d
   actions-runner-linux-x64-2.317.0.tar.gz" | shasum -a 256 -c
   ```

4. **Extract the Installer:**

   ```
   tar xzf actions-runner-linux-x64-2.317.0.tar.gz
   ```

5. **Configure the Runner:**

   ```
   ./config.sh --url https://github.com/xxxxxxxxxxx/Repo-2 --token
   xxxxxxxxxxxxxxx
   ```

   1. **Runner Group:** By default, no runner group is created. You can use the default group.
   2. **Runner Name:** Enter a name for the runner (e.g., `Runner-1`).
   3. **Labels:** Optionally, add labels to help identify where the runner should execute jobs.
   4. **Work Folder:** The default work folder is `_work`. You can change this if desired.

Setting Up Maven, Trivy, and GitHub Actions Runner

1. **Install Maven:** Ensure Maven is installed on the machine:

   ```
   sudo apt install maven -y
   ```

2. **Set Up Trivy:** Install Trivy by adding its repository and then installing the package:

```
# Install dependencies
sudo apt-get install wget apt-transport-https gnupg lsb-
release

# Add the Trivy repository key
wget -qO - https://aquasecurity.github.io/trivy-
repo/deb/public.key | sudo apt-key add -

# Add the Trivy repository
echo deb https://aquasecurity.github.io/trivy-repo/deb
$(lsb_release -sc) main | sudo tee -a
/etc/apt/sources.list.d/trivy.list

# Update the package list and install Trivy
sudo apt-get update
sudo apt-get install trivy
```

6. **Start the Runner:**

   ```
   ./run.sh
   ```

1. **Check the Runner Status:** After starting the runner, check the GitHub Actions settings. Initially, the runner may appear as "offline." To bring it online, ensure the `run.sh` script is running.

Meanwhile on the machine used to create EKS cluster check the creation of the EKS cluster

To ensure we can access it, we need to update the kubeconfig file. Use the following command:

```
aws eks --region <region> update-kubeconfig --name <cluster_name>
```

Replace `<region>` with your AWS region (e.g., `ap-south-1`) and `<cluster_name>` with your EKS cluster name. This command updates the kubeconfig file used for authentication.

Next, verify that `kubectl` is installed. If it's not installed, you can install it using Snap:

```
sudo snap install kubectl --classic
```

After installation, check the cluster status with:

```
kubectl get nodes
```

This will show the nodes in your cluster and their status. If the nodes are in the "Ready" state, your cluster is up and running correctly.

Now, we can get started with writing the CI/CD pipeline.

Step-by-Step Instructions

1. **Navigate to GitHub Actions:**

- Go to the Actions tab in your GitHub repository.

2. **Choose a Template:**
   - You'll find many templates to help you get started. For this example, select "Java with Maven" and click on "Configure".
3. **Edit the Workflow File:**
   - The default configuration will create a pipeline for the `main` branch. We want to set it up for the `Dev` branch instead.
   - Copy the provided configuration.
   - Navigate to the `Dev` branch and create a new file at `.github/workflows/cicd.yaml`.
4. **Paste and Edit Configuration:**
   - Paste the copied configuration into the new file. Here is the configuration:

```yaml
name: CICD

on:
  push:
    branches: ["Dev"]
  pull_request:
    branches: ["Dev"]

jobs:
  build:
    runs-on: self-hosted

    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven
      - name: Build with Maven
        run: mvn package --file pom.xml
```

Explanation of the Configuration:

- **Workflow Name:**
  - `name: CICD` specifies the name of the workflow.
- **Triggers:**
  - The workflow triggers on pushes and pull requests to the `Dev` branch.
- **Jobs:**
  - The `build` job runs on a self-hosted runner.
- **Steps:**
  - **Checkout Repository:** Uses `actions/checkout@v4` to check out the repository code to the runner.
  - **Set up JDK 17:** Uses `actions/setup-java@v4` to set up JDK 17 with the Temurin distribution and enables caching for Maven dependencies.
  - **Build with Maven:** Runs the `mvn package` command, specifying the `pom.xml` file.

Additional Note:

One interesting part of GitHub Actions is that you don't need to specifically install multiple tools on the virtual machine. You can directly call the action in your pipeline and utilize it. For example, we haven't installed Java on our virtual machine, but using the setup-java action, we can directly use the required Java version in our pipeline.

Commit and Push Changes:

- Save and commit the changes.
- Push the changes to the `Dev` branch.

Verify the Pipeline:

1. **Go to Actions:**
   - Navigate to the Actions tab in your GitHub repository.
2. **Monitor the Pipeline:**
   - You should see the pipeline has started. Click on it to monitor the stages and verify that they are running correctly.
3. **Check Success:**
   - Once complete, verify that the application has been built successfully.

Commit and Push Changes:

- Save and commit the changes.
- Push the changes to the `Dev` branch.

Verify the Pipeline:

1. **Go to Actions:**
   - Navigate to the Actions tab in your GitHub repository.
2. **Monitor the Pipeline:**
   - You should see the pipeline has started. Click on it to monitor the stages and verify that they are running correctly.
3. **Check Success:**
   - Once complete, verify that the application has been built successfully.

Step-by-Step Instructions to Extend the CI/CD Pipeline

Now, we need to add multiple stages in our pipeline. We'll add a file system scan using Trivy and a SonarQube analysis.

1. Navigate to the Dev Branch and Edit Workflow File:

- Go to the `Dev` branch and edit the `.github/workflows/cicd.yaml` file.

2. Add Trivy File System Scan Stage:

- Copy the existing build stage configuration.
- Rename the new stage to `Trivy FS Scan`.
- Configure the Trivy scan command.

Here is the updated configuration:

```
name: CICD
on:
  push:
    branches:
      - Dev
  pull_request:
    branches:
      - Dev
jobs:
  build:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: "17"
          distribution: temurin
          cache: maven
      - name: Build with Maven
        run: mvn package --file pom.xml
      - name: Trivy FS Scan
        run: trivy fs --format table -o fs.html .
```

Adding SonarQube Analysis to the CI/CD Pipeline

To perform SonarQube analysis, we need to ensure that a SonarQube server is set up. For simplicity, we'll set up the SonarQube server directly on the runner using a Docker container.

- Install Docker on the runner.

```
Sudo apt install docker.io -y
sudo chmod 666 /var/run/docker.sock
```

- Run a SonarQube Docker container.

```
sudo docker run -d -p 9000:9000 sonarqube:lts-community
```

Setting Up SonarQube Analysis in the CI/CD Pipeline

Usually, when working with SonarQube, we need two things:

1. **SonarQube Server**: This is where the reports are published and analyzed.
2. **SonarQube Scanner**: This performs the analysis, generates the report, and publishes it to the server.

We've already set up the SonarQube server. Now, we need to set up the SonarQube scanner. The benefit of using GitHub Actions is that we can simply call an action to set up the SonarQube scanner.

First, let access SonarQube server, which will be running on port 9000, wait for it to start.

Setting Up SonarQube Server

To set up the SonarQube server, follow these steps:

1. **Access the SonarQube Server**: Copy the public IP of the machine where the SonarQube Docker container is running. Open your browser and paste the IP followed by `:9000`.
2. **Change Default Credentials**: By default, SonarQube uses the username `admin` and password `admin`. Log in with these credentials, then change the password to something secure of your choice. Click on "Update" to save the new password.

Meanwhile, define the stages in our pipeline we can utilize different actions by calling them directly in pipeline configuration. Here, we are using the action sonar-scanner which sets up the SonarQube scanner to perform the scanning.

Adding SonarQube Analysis to the CI/CD Pipeline

To perform SonarQube analysis in our CI/CD pipeline, we need to add the necessary steps for SonarQube. Here's how to set it up:

1. **Add Secrets for SonarQube**:
   - **SonarQube Host URL**: Go to your repository settings, navigate to Secrets and variables > Actions, and add a new repository secret. Name it `SONAR_HOST_URL` and set its value to the URL of your SonarQube server (without the trailing slash).
   - **SonarQube Token**: Generate a token from your SonarQube server by going to Administration > Security > Users > Administrator. Create a token, copy it, and add it as a new repository secret named `SONAR_TOKEN`.
2. **Update Your Workflow Configuration**:
   - Add the SonarQube scan stage to your pipeline configuration as follows:

```
name: CICD

on:
  push:
    branches:
      - Dev
  pull_request:
    branches:
      - Dev

jobs:
  build:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: "17"
```

```
          distribution: temurin
          cache: maven
      - name: Build with Maven
        run: mvn package --file pom.xml
      - name: Trivy FS Scan
        run: trivy fs --format table -o fs.html .
      - name: SonarQube Scan
        uses: sonarsource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
```

Explanation

- **Set up Secrets**: Ensure the `SONAR_HOST_URL` and `SONAR_TOKEN` are added as secrets in your repository settings.
- **Workflow Triggers**: The pipeline triggers on pushes and pull requests to the `Dev` branch.
- **Jobs**: The `build` job runs on a self-hosted runner and includes steps to check out the repository, set up JDK 17, build with Maven, perform a Trivy FS scan, and execute a SonarQube scan

Adding Project Details for SonarQube Analysis

To run a SonarQube analysis, you need to provide information about the project name, project key, and the location of Java binaries. This information is specified in a `sonar-project.properties` file in your repository. Here's how you can set this up:

1. **Create/Edit `sonar-project.properties`**:
   - Navigate to the `dev` branch of your repository.
   - Create or edit the `sonar-project.properties` file and add the following details:

```
sonar.projectKey=TaskMaster
sonar.projectName=TaskMaster
sonar.java.binaries=target
```

Committing the Changes and Monitoring the Pipeline

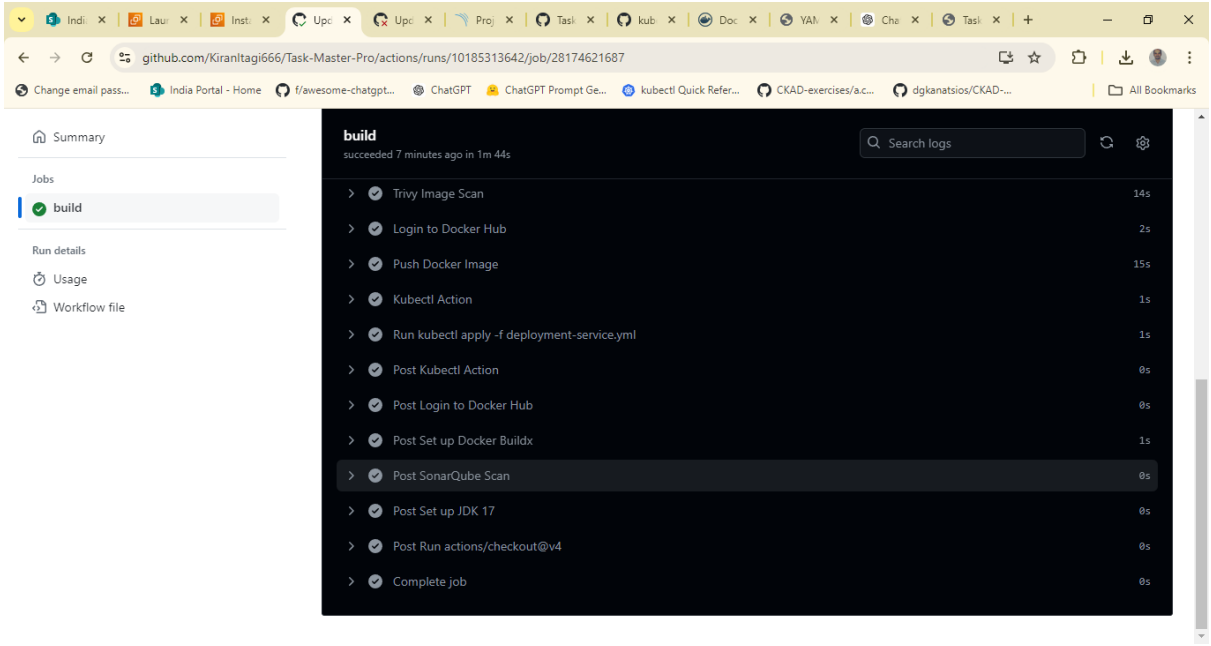
1. **Commit the Changes**:
   - Ensure all changes, including the `sonar-project.properties` file and the workflow configuration, are staged for commit.
   - Commit the changes with a meaningful commit message, e.g., "Add SonarQube configuration and update CI/CD pipeline".
2. **Monitor the Pipeline**:
   - Navigate to the **Actions** tab in your GitHub repository.
   - You should see the newly triggered pipeline listed.
   - Click on the pipeline to monitor the progress of each stage.

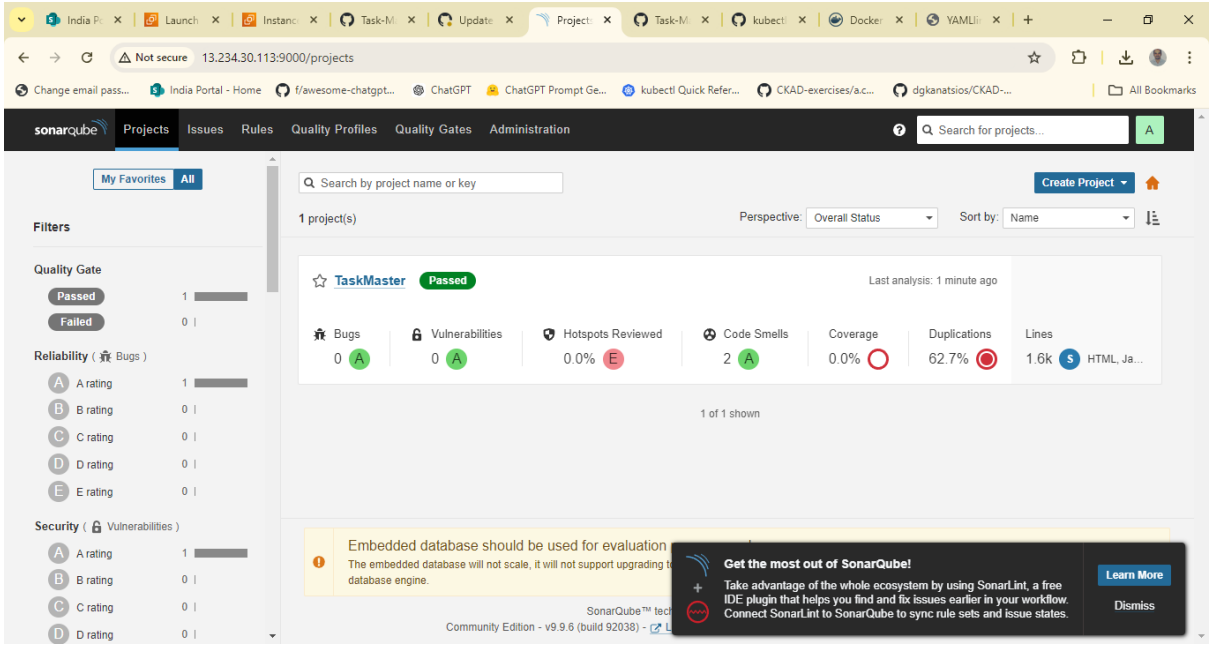
Here's a summary of the changes that were committed:

```
name: CICD

on:
  push:
    branches:
      - Dev
  pull_request:
    branches:
      - Dev

jobs:
  build:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: "17"
          distribution: temurin
          cache: maven
      - name: Build with Maven
        run: mvn package --file pom.xml
      - name: Trivy FS Scan
        run: trivy fs --format table -o fs.html .
      - name: SonarQube Scan
        uses: sonarsource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
```

Monitoring Steps

1. **Navigate to the Actions Tab**:
   - Go to your GitHub repository.
   - Click on the **Actions** tab.
2. **Select the Pipeline**:
   - You should see the pipeline that was triggered by your recent commit.
   - Click on the pipeline to view the detailed progress of each stage.
3. **Check Each Stage**:
   - Verify that each stage runs successfully:
     - **Checkout**: The repository is checked out.
     - **Set up JDK 17**: JDK 17 is installed.
     - **Build with Maven**: The Maven build runs and completes.
     - **Trivy FS Scan**: The filesystem scan is performed.
     - **SonarQube Scan**: The SonarQube analysis is executed.
4. **Review Results**:
   - Ensure each step completes without errors.
   - For the SonarQube scan, you can visit your SonarQube server to review the detailed analysis results.





Adding Docker Stages to the CI/CD Pipeline

To extend your CI/CD pipeline to include Docker image building, scanning, and pushing, follow these steps:

1. **Explore GitHub Marketplace Actions:** To simplify the process of building and pushing Docker images, you can use pre-defined actions available on the GitHub Marketplace. For example, you can use the Build and Push Docker Images action to handle these tasks. You can search for the actions you need, and the stages are already available for use; you just need to copy and paste them.
2. **Add Docker Stages to Your Pipeline:** Integrate the following Docker stages into your existing pipeline configuration. This will include steps for setting up QEMU,

Docker Buildx, logging into Docker Hub, building the Docker image, scanning it with Trivy, and pushing it to Docker Hub.

(Make sure to adjust the Docker image name and tags as per your specific configuration or requirement.)

Update your `cicd.yaml` file as follows:

```
name: CICD

on:
  push:
    branches:
      - Dev
  pull_request:
    branches:
      - Dev

jobs:
  build:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: "17"
          distribution: temurin
          cache: maven
      - name: Build with Maven
        run: mvn package --file pom.xml
      - name: Trivy FS Scan
        run: trivy fs --format table -o fs.html .
      - name: SonarQube Scan
        uses: sonarsource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Build Docker Image
        run: |
          docker build -t kviondocker/devtaskmaster:latest .
      - name: Trivy Image Scan
        run: trivy image --format table -o image.html
kviondocker/devtaskmaster:latest
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Push Docker Image
        run: |
          docker push kviondocker/devtaskmaster:latest
```

Explanation of Docker Stages:

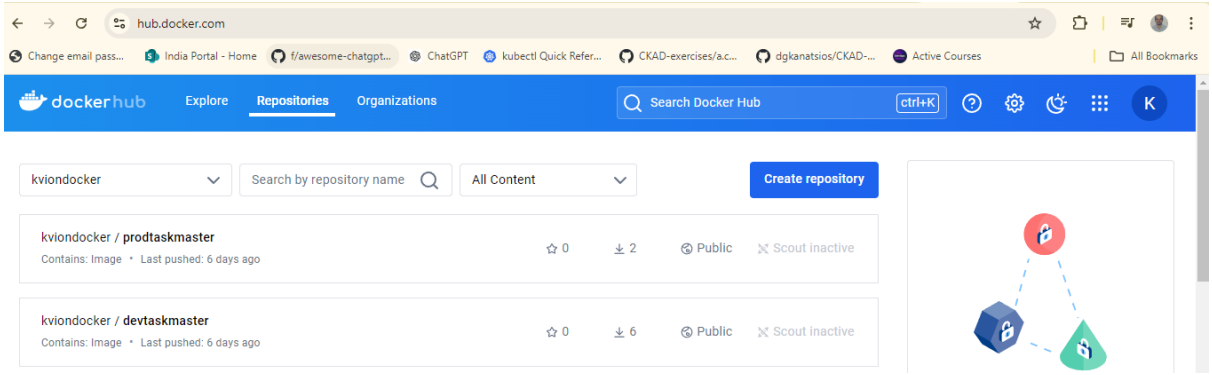
- **Set up QEMU**: Configures QEMU for emulating different architectures.
- **Set up Docker Buildx**: Prepares Docker Buildx for advanced image building.
- **Build Docker Image**: Builds the Docker image and tags it as `kviondocker/devtaskmaster:latest`. **Adjust the image name and tag as per your configuration or requirement.**
- **Trivy Image Scan**: Scans the Docker image for vulnerabilities and outputs the results to `image.html` using the command `trivy image --format table -o image.html kviondocker/devtaskmaster:latest`.
- **Login to Docker Hub**: Authenticates to Docker Hub using credentials stored in GitHub Secrets.
- **Push Docker Image**: Pushes the Docker image to Docker Hub.

3. **Set Up Secrets:** Ensure that the following secret variables are set up in your GitHub repository:
   - **DOCKERHUB_USERNAME**: Your Docker Hub username.
   - **DOCKERHUB_TOKEN**: Your Docker Hub token (password).

   To set these secrets:

   - Go to your repository's **Settings**.
   - Scroll down to **Secrets and variables** and select **Actions**.
   - Add new repository secrets for **DOCKERHUB_USERNAME** and **DOCKERHUB_TOKEN**.

By following these steps, you'll extend your CI/CD pipeline to handle Docker image management, ensuring that your Docker images are built, scanned for vulnerabilities, and deployed as part of your automated workflow.



To deploy the application to your Kubernetes cluster, follow these steps:

1. **Access the Kubernetes Configuration File:**
   - Navigate to the `.kube` directory on your master node using:

     ```
     cd ~/.kube/
     ```

   - Display the content of the `config` file:

```
cat config
```

- This file contains the configuration required for authenticating with your Kubernetes cluster.

2. **Encode the Kubernetes Configuration File:**
   - Convert the `config` file to base64 format using the following command:

```
base64 ~/.kube/config
```

3. **Create a Kubernetes Secret:**
   - Go to your GitHub repository settings.
   - Navigate to "Secrets and variables" under GitHub Actions.
   - Add a new secret with the name `k8s_kube_config` and paste the base64-encoded content from the previous step as its value.
4. **Use the Secret in Your Pipeline:**
   - You can then use this secret in your CI/CD pipeline to configure `kubectl` for deploying your application to the Kubernetes cluster.

To add the final stage for deploying to Kubernetes, follow these steps:

1. **Add Kubectl Action to Your Pipeline:**
   - Explore GitHub Marketplace Actions to find pre-defined actions for `kubectl`. For example, you can use the [Kubectl Action]() for managing Kubernetes deployments.
   - Incorporate the following steps into your pipeline:

```
- name: Kubectl Action
  uses: tale/kubectl-action@v1
  with:
    base64-kube-config: ${{ secrets.KUBE_CONFIG }}
- run: |
    kubectl apply -f deployment-service.yml
```

2. **Update Docker Image in Deployment YAML:**
   - Navigate to the `deployment-service.yml` file in your `Dev` branch.
   - Update the Docker image name to reflect the new image:

```
image: kviondocker/devtaskmaster:latest
```

(Make sure to adjust the Docker image name and tags as per your specific configuration or requirement)

3. **Commit and Push Changes:**
   - Commit and push the changes to your repository. This action will trigger the pipeline, deploying your application to the Kubernetes cluster.
4. **Monitor the Pipeline:**
   - After committing, the pipeline will start. Monitor it to ensure that the deployment proceeds as expected.

Final Pipeline Configuration:

```
name: CICD
on:
  push:
    branches:
      - Dev
  pull_request:
    branches:
      - Dev
jobs:
  build:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: "17"
          distribution: temurin
          cache: maven
      - name: Build with Maven
        run: mvn package --file pom.xml
      - name: Trivy FS Scan
        run: trivy fs --format table -o fs.html .
      - name: SonarQube Scan
        uses: sonarsource/sonarqube-scan-action@master
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
          SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}
      - name: Set up QEMU
        uses: docker/setup-qemu-action@v3
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3
      - name: Build Docker Image
        run: |
          docker build -t kviondocker/devtaskmaster:latest .
      - name: Trivy Image Scan
        run: trivy image --format table -o image.html
kviondocker/devtaskmaster:latest
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Push Docker Image
        run: |
          docker push kviondocker/devtaskmaster:latest
      - name: Kubectl Action
        uses: tale/kubectl-action@v1
        with:
          base64-kube-config: ${{ secrets.KUBE_CONFIG }}
      - run: |
          kubectl apply -f deployment-service.yml
```

After the pipeline completes successfully, you can verify the deployment with the following steps:

1. **Check Service Status:**
   - On the Master node, run the command:

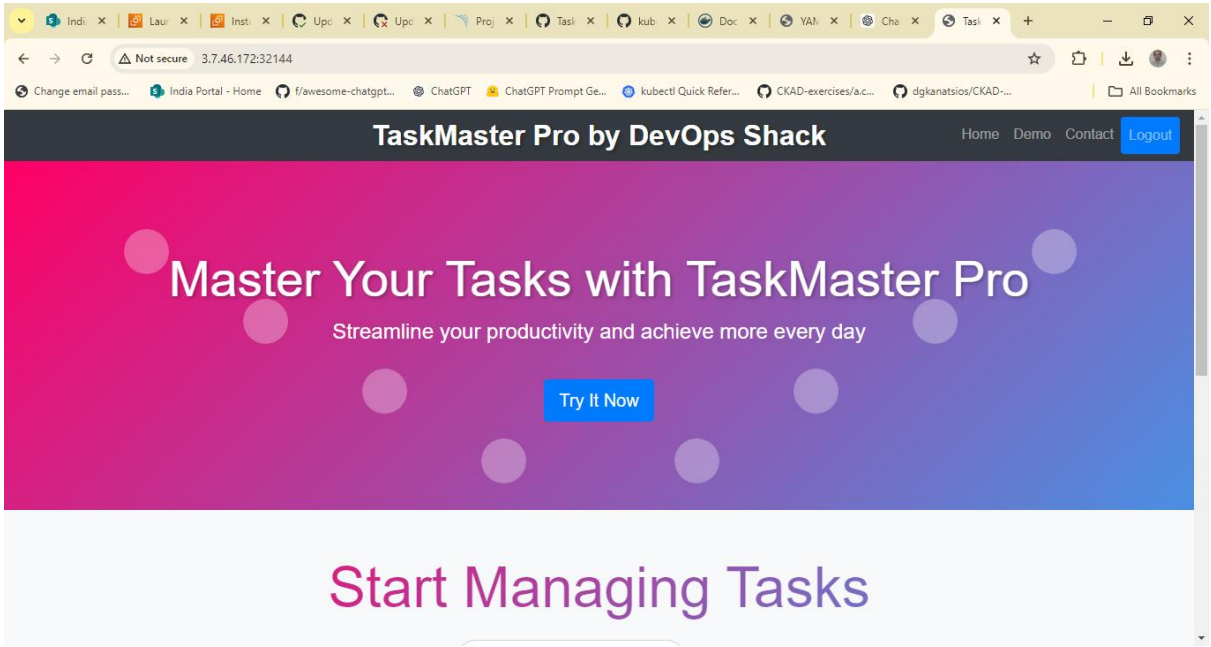
     ```
     kubectl get svc
     ```

   - Look for your application, such as `Taskmaster svc`, to confirm it is created.
2. **Access the Application:**
   - Find the IP address of the worker node and the port assigned to your service. For example, if the worker node IP is `X.X.X.X` and the port is `32560`, you can access your application using:

     ```
     http://X.X.X.X:32560
     ```

   - Open this URL in your browser to see if the application is deployed correctly.

By following these steps, you can confirm that your application is successfully deployed and accessible.



<u>To deploy the application to the Production (Master) cluster, follow these steps and updates based on the configuration:</u>

1. **Prepare Kubernetes Configuration:**
   - **Access the EKS Cluster:**
     - On the EKS cluster server, navigate to the Kubernetes configuration directory:

       ```
       cd ~/.kube/
       ```

     - Display the contents of the config file:

```
cat config
```

- Encode the config file in base64:

```
base64 ~/.kube/config
```

- **Create a GitHub Secret:**
  - Go to GitHub Settings > Secrets and Variables > Actions > New repository secret.
  - Name: `KUBE_CONFIG_PROD`
  - Value: Paste the base64-encoded config file.
  - Click "Add secret."

2. **Update Docker Image Tag:**
   - **Modify the Docker Image Tag:**
     - In the `deployment-service.yml` file, update the Docker image name to the new tag:

```
yaml
Copy code
image: kviondocker/prodtaskmaster:latest
```

3. **Set Up AWS CLI on the Runner:**
   - **Install AWS CLI:**
     - Execute the following command on the runner to install AWS CLI:

```
bash
Copy code
curl "https://d1uj6qtbmh3dt5.cloudfront.net/awscli-exe-
linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

   - **Configure AWS CLI:**
     - Run the AWS configure command and provide your credentials:

```
aws configure
```

     - Enter your AWS Access Key ID, Secret Access Key, and default region (e.g., `ap-south-1`).

4. **Create and Push the Production Docker Image:**
   - **Update and Push Docker Image:**
     - Add a new job to the pipeline for pushing the tagged Docker image and deploying to the EKS cluster.

Pipeline Configuration:

```
name: CDPROD

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: self-hosted

    steps:
      - name: Build with Maven
        run: mvn -B package --file pom.xml

      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: Pull and Tag Docker Image
        run: |
          docker pull kviondocker/devtaskmaster:latest
          docker tag kviondocker/devtaskmaster:latest
kviondocker/prodtaskmaster:latest
          docker push kviondocker/prodtaskmaster:latest

      - name: Kubectl Action
        uses: tale/kubectl-action@v1
        with:
          base64-kube-config: ${{ secrets.KUBE_CONFIG_PROD }}

      - run: |
          kubectl apply -f deployment-service.yml
```

Summary:

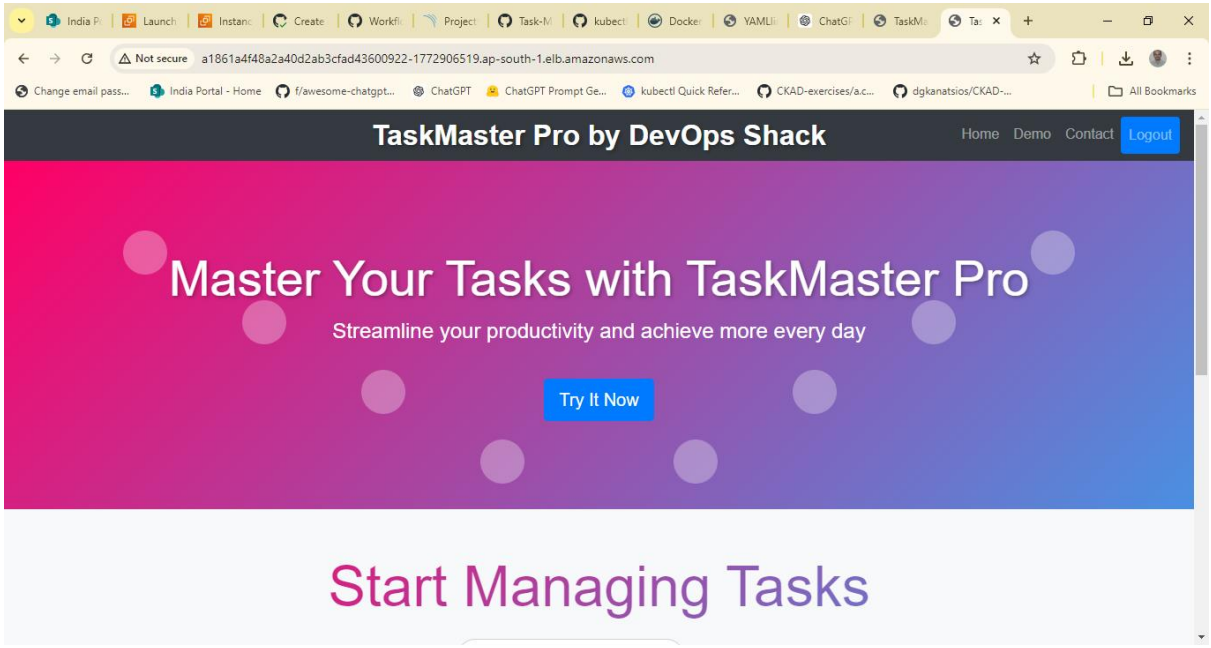
- **Prepare Kubernetes Configuration:** Encode and add the base64-configured file to GitHub secrets.
- **Update Docker Image Tag:** Modify the deployment configuration with the new image tag.
- **Set Up AWS CLI:** Install and configure AWS CLI on the runner.
- **Pipeline Configuration:** Use the GitHub Actions pipeline to handle Docker image tagging, pushing, and Kubernetes deployment.

After committing these changes, monitor the pipeline to ensure the deployment is successful and the application is running as expected in the Production cluster.

You can see that the deployment and service have been created, and the workflow has completed successfully.

To verify the deployment:

1. **Check Deployment Status:**
   - On your Terraform server, run the command:

     ```
     kubectl get all
     ```

   - This will show that the application is deployed.
2. **Access the Application:**
   - Copy the external IP address and port from the service details.
   - Open a browser and navigate to `http://<external-ip>:<port>`.
   - You should be able to see your application running.